B3 - Base de Données NoSQL

LEARNHUB

Plateforme E-Learning

Wafae ERRAI

M. Oussama ETTALALI

03 Mars 2026

Table des matières

1 Introduction **2**

2 Phase 1 : Modélisation et Données **3**
2.1 Architecture de la Base de Données . . . 3
2.1.1 Collections Principales . . . 3
2.2 Script de Seed (Peuplement) . . . 3

3 Phase 2 : Requêtes et Manipulation **5**
3.1 Requêtes Complexes Implémentées . . . 5
3.1.1 Filtrage Avancé . . . 5
3.1.2 Mise à jour Atomique . . . 5
3.1.3 Recherche Textuelle (Regex) . . . 5

4 Phase 3 : Agrégation et Analyse **7**
4.1 Statistiques Globales (Bonus) . . . 7
4.2 Analyses Métier . . . 7

5 Phase 4 : Backend API Node.js **9**
5.1 Structure de l'API . . . 9
5.1.1 Gestion des Cours (`GET /api/courses`) . . . 9
5.1.2 Gestion des Inscriptions (`POST /api/enroll`) . . . 9
5.2 Export de Données (Bonus) . . . 9

6 Phase 5 : Interface Frontend & Bonus **10**
6.1 Fonctionnalités Clés . . . 10
6.2 Postman Collection . . . 10

7 Conclusion **12**

Chapitre 1

Introduction

Le projet **LearnHub** consiste en la conception, le développement et le déploiement local d'une plateforme d'apprentissage en ligne (LMS). L'objectif principal était de démontrer une maîtrise approfondie de **MongoDB** et de son intégration avec un backend **Node.js**.

Ce rapport détaille les choix techniques, l'architecture de la base de données, les algorithmes d'agrégation complexes mis en place, et l'interface utilisateur développée pour interagir avec l'API. Nous avons couvert l'intégralité du cahier des charges, incluant toutes les phases obligatoires ainsi que les fonctionnalités bonus (Frontend complet, Recherche Regex, Statistiques Globales, Export JSON).

Chapitre 2

Phase 1 : Modélisation et Données

2.1 Architecture de la Base de Données

Une réflexion approfondie a été menée pour choisir entre l'imbrication (*Embedding*) et le référencement (*Referencing*), afin d'optimiser les performances de lecture tout en maintenant la cohérence des données.

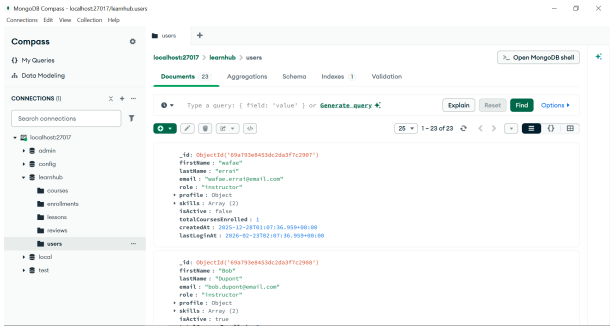

FIGURE 2.1 – Choix de MongoDB pour sa flexibilité de schéma.

2.1.1 Collections Principales

- **Users** : Stocke les étudiants et les instructeurs. *Choix :* Nous avons utilisé l'**imbrication** pour le champ `profile` (skills, bio) car ces données sont toujours affichées avec l'utilisateur.
- **Courses** : Le cœur de l'application. *Choix :* Les métadonnées (catégorie, difficulté) sont imbriquées. Cependant, pour éviter d'atteindre la limite de 16MB par document, les **Lessons** (leçons) et **Reviews** (avis) sont liées via des références ou gérées dans des tableaux optimisés.
- **Enrollments** : Table de liaison entre `Users` et `Courses`. *Choix :* Utilisation du **référencement** pour permettre une scalabilité infinie du nombre d'inscriptions par cours.

2.2 Script de Seed (Peuplement)

Un script JavaScript (`seed.mongosh.js`) a été développé pour initialiser la base de données avec des données réalistes. Il génère :

— **20 Utilisateurs** avec des rôles distincts (Student, Instructor, Admin).
— **15 Cours** répartis dans diverses catégories (DevOps, Data Science, Web).
— Des **Logs d'activité** pour tracer les actions importantes.

```
PS C:\Users\wafae\OneDrive\Bureau\Projet Final - LearnHub> .\mongosh.exe "mong
odb://localhost:27017/learnhub" scripts/seed.mongosh.js
Base de données nettoyée. Début de l'insertion...
20 utilisateurs insérés.
15 cours insérés.
45 leçons insérées.
30 inscriptions insérées.
22 avis insérés.
Initialisation terminée avec succès !
PS C:\Users\wafae\OneDrive\Bureau\Projet Final - LearnHub>
```

FIGURE 2.2 – Exécution réussie du script de chargement des données.

Chapitre 3

Phase 2 : Requêtes et Manipulation

Cette phase valide la capacité à interroger la base de données avec le langage de requête MongoDB (MQL).

3.1 Requêtes Complexes Implémentées

Nous avons implémenté et testé plus de 30 requêtes différentes dans le fichier `queries.mongosh.js`, couvrant :

3.1.1 Filtrage Avancé

Utilisation des opérateurs logiques et de comparaison pour extraire des données précises :

- `$in` : Pour trouver des cours dans plusieurs catégories spécifiques (ex : 'Web' ou 'Mobile').
- `$gte` / `$lte` : Pour filtrer les cours par fourchette de prix.
- `$or` / `$and` : Pour des conditions combinées (ex : Cours $>$ 4 étoiles ET publiés après 2023).

3.1.2 Mise à jour Atomique

Utilisation des opérateurs de mise à jour pour garantir l'intégrité des données sans concurrence :

- `$inc` : Pour incrémenter le nombre d'inscrits lors d'un achat.
- `$push` : Pour ajouter un tag à un cours sans écraser le tableau existant.
- `$set` : Pour modifier partiellement un document (ex : changer le statut d'un utilisateur).

3.1.3 Recherche Textuelle (Regex)

Implémentation de recherches insensibles à la casse pour trouver des cours par mot-clé dans le titre ou la description :

```
db.courses.find({
    title: { $regex: /python/i }
})
```

```
PS C:\Users\wafae\OneDrive\Bureau\Projet Final - LearnHub> .\mongosh.exe "mongodb://localhost:27017/learnhub" sc
ripts/queries.mongosh.js
=== 2.1 OPÉRATIONS CRUD ===
1. Insert User
2. Insert 3 Courses
3. Update City
4. Increment Enrollment
5. Push Skill
6. Pull Tag
7. Deactivate Inactive Users
8. Upsert Profile
9. Delete Review
10. Delete Cancelled Enrollments
=== Fin des opérations CRUD ===

=== 2.2 SÉLECTION & FILTRES ===
11. Find Courses Price Range [20, 80]
Trouvé: 13 cours
12. Find Categories Database OR Web
Trouvé: 9 cours
13. Find Difficulty NOT Advanced
Trouvé: 13 cours
14. Find Active Students
Trouvé: 16 étudiants actifs
15. Find Free OR High Rated Courses
Trouvé: 8 cours
16. Find Reviews with UpdatedAt
Trouvé: 1 avis mis à jour
17. Find Users in Paris
Trouvé: 3 parisiens
18. Find Published AND High Rated (Explicit AND)
Trouvé: 12 cours
19. Find Enrollments Not Cancelled NOR Paused
Trouvé: 12 inscriptions valides
20. Project Title and Price
[
  { title: 'Maîtriser React', price: 34.27 },
  { title: 'Python Avancé', price: 11.04 },
  { title: 'Node.js API REST', price: 0 }
]
21. Project All Except Profile
22. Top 5 Rated Courses
Top 1: Node.js API REST
23. Sort Courses by Price Ascending
Moins cher: 0
24. Page 2 of Courses
Page 2 contient 10 cours
25. Count Published Courses
Total publiés: 18
=== Fin de la Sélection ===

=== 2.3 LOGIQUE MÉTIER & TRANSACTIONS (Simulation) ===
26. Enrollment Logic (Simulation)
Inscription réalisée avec succès
27. Complex Catalog Query
Catalogue Web Top: 2 résultats
28. Progress Update
Progression mise à jour
29. Cascade Delete Logic
Cours 69a60aad0c6e691c3c7c291c supprimé en cascade.
```

FIGURE 3.1 – Exécution des requêtes mongoDB dans le terminal.

Chapitre 4

Phase 3 : Agrégation et Analyse

La puissance de MongoDB réside dans son Framework d'Agrégation, utilisé ici pour générer des statistiques analytiques en temps réel.

4.1 Statistiques Globales (Bonus)

Nous avons créé un pipeline d'agrégation exposé via l'endpoint API `/api/stats` qui calcule en une seule requête :

— Le nombre total d'utilisateurs inscrits.
— Le nombre total de cours disponibles.
— Le volume total des inscriptions (Chiffre d'affaires potentiel).

4.2 Analyses Métier

Des pipelines complexes ont été écrits pour répondre à des questions business :

1. **Prix Moyen par Catégorie** : Utilisation de `$group` et `$avg` pour déterminer les secteurs les plus lucratifs.
2. **Top Instructeurs** : Classement des instructeurs ayant le plus de cours publiés via `$sortByCount`.
3. **Cours les Mieux Notés** : Filtrage (`$match`) et tri (`$sort`) des cours ayant une moyenne supérieure à 4.5.

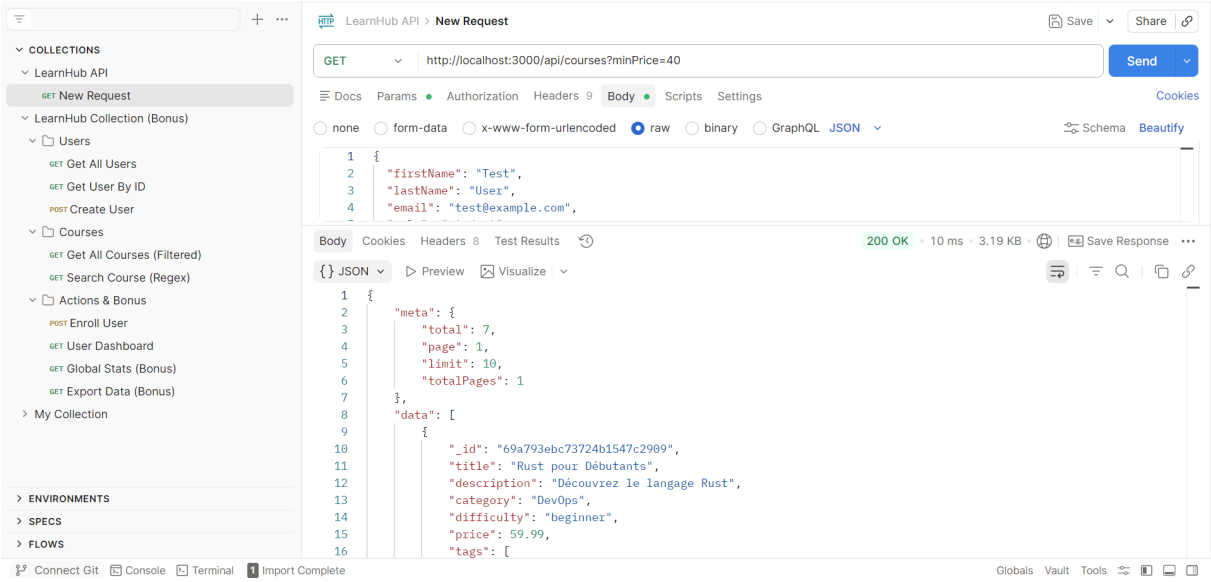

FIGURE 4.1 – Résultat JSON (Postman) : Affichage complet de la liste des cours (API Phase 4).

Chapitre 5

Phase 4 : Backend API Node.js

Le backend assure la communication entre la base de données et le client. Il a été développé avec **Express.js** et le **driver natif MongoDB**.

5.1 Structure de l'API

L'API est structurée autour de ressources RESTful :

5.1.1 Gestion des Cours (`GET /api/courses`)

Cet endpoint est le plus riche. Il supporte :
— **Pagination** : Paramètres `page` et `limit`.
— **Tri** : Par prix (croissant/décroissant) ou date.
— **Filtres** : Par catégorie.
— **Recherche (Nouveau)** : Recherche textuelle via Regex sur le titre.

5.1.2 Gestion des Inscriptions (`POST /api/enroll`)

Cet endpoint transactionnel assure l'intégrité des données :

1. Vérifie si l'utilisateur existe.
2. Vérifie si le cours existe.
3. Vérifie si l'utilisateur n'est pas déjà inscrit (doublon interdit).
4. Crée le document `Enrollment`.
5. Incrémente le compteur `studentsCount` du cours (Atomicité).

5.2 Export de Données (Bonus)

Un endpoint spécial `GET /api/export` a été ajouté. Il génère un dump complet de la base de données au format JSON, permettant aux administrateurs de sauvegarder l'état du système.

Chapitre 6

Phase 5 : Interface Frontend & Bonus

Pour offrir une expérience utilisateur complète, une interface web moderne, responsive et fonctionnelle a été développée sans framework lourd (Vanilla JS / CSS), garantissant des performances maximales.

6.1 Fonctionnalités Clés

- **Catalogue Dynamique** : Les cartes de cours sont générées via JavaScript en consommant l'API.
- **Barre de Recherche et Filtres** : L'utilisateur peut rechercher un cours par nom (ex : "Python") ou par catégorie. L'interface se met à jour sans rechargement de page (AJAX).
- **Tableau de Bord Étudiant** : Une section personnalisée affiche les cours auxquels l'utilisateur est inscrit.
- **Statistiques en Temps Réel** : Un bandeau affiche le nombre total d'utilisateurs et de cours, récupéré via l'agrégation backend.
- **Export JSON** : Un bouton direct permet de télécharger les données.

6.2 Postman Collection

Pour faciliter la correction et les tests, une collection Postman complète (`learnhub_collection.json`) a été générée. Elle contient toutes les requêtes pré-configurées pour tester chaque fonctionnalité du projet.

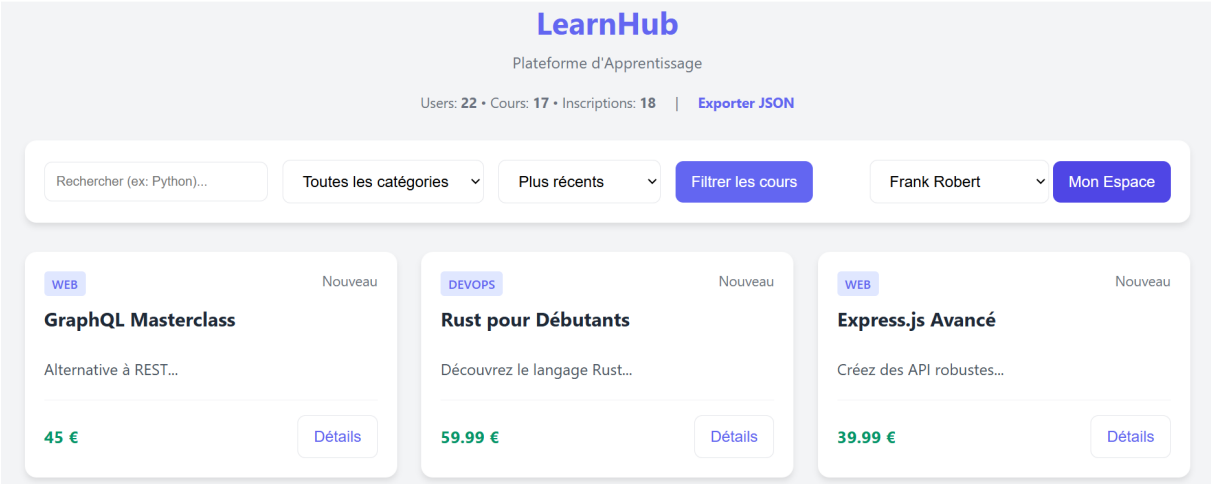

FIGURE 6.1 – Interface LearnHub v1.0 avec recherche et statistiques.

Chapitre 7

Conclusion

Le projet LearnHub a été un exercice complet couvrant tout le cycle de vie d'une application orientée données. Nous avons réussi à :

1. Modéliser une base de données NoSQL performante.
2. Construire une API robuste capable de gérer des transactions complexes.
3. Développer un frontend intuitif exploitant ces données.